



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
Campus Maracanaú
Bacharelado em Ciência da Computação

Matheus Castelo, Breno Vinicius, Sofia Prata, Vinicius Kainan

SIGEJ: SISTEMA DE GESTÃO DE JARDINAGEM E MANUTENÇÃO
Relatório Técnico de Desenvolvimento

Maracanaú - CE
Dezembro/2025

Matheus Castelo, Breno Vinicius, Sofia Prata, Vinicius Kainan

SIGEJ: SISTEMA DE GESTÃO DE JARDINAGEM E MANUTENÇÃO

Relatório apresentado à disciplina de Banco de Dados / Programação, do curso de Bacharelado em Ciência da Computação, como requisito parcial para obtenção de nota.

Maracanaú - CE
Dezembro/2025

Sumário

1	Introdução	4
2	Tecnologias Utilizadas	4
3	Modelagem do Banco de Dados	4
4	Implementação do Backend e API	5
5	Resultados e Relatórios Gerenciais	8
5.1	Execução das Consultas	9
6	Conclusão	13

1 Introdução

Este relatório descreve o desenvolvimento do ****SIGEJ**** — Sistema de Informação de Gestão de Jardinagem e Manutenção. O sistema foi projetado para atender às necessidades da Coordenação de Infraestrutura do Campus Maracanaú, visando o controle de ordens de serviço, equipes de manutenção e estoque de materiais.

O desenvolvimento seguiu rigorosamente os requisitos estipulados, destacando-se a obrigatoriedade do uso de comandos SQL explícitos para todas as operações de banco de dados, sendo vedado o uso de ferramentas de Mapeamento Objeto-Relacional (ORM).

2 Tecnologias Utilizadas

Para a implementação da solução, foram selecionadas tecnologias robustas e adequadas ao escopo acadêmico e profissional do projeto:

- **Linguagem de Programação:** C# (.NET) para o desenvolvimento do backend, garantindo tipagem forte e alto desempenho.
- **Banco de Dados:** PostgreSQL, gerenciado através da ferramenta DataGrip.
- **Documentação de API:** Swagger (OpenAPI) para visualização e teste dos endpoints.
- **Testes Automatizados:** Python, utilizado para criar scripts que validam os endpoints e o fluxo de dados do sistema.

3 Modelagem do Banco de Dados

O banco de dados foi estruturado contendo 25 tabelas relacionais, cobrindo entidades essenciais como pessoa, funcionário, equipe de manutenção, estoque e ordem de serviço. A estrutura foi implementada no PostgreSQL e gerenciada via DataGrip, garantindo a integridade referencial e a normalização dos dados.

As Figuras 1 e 2 apresentam o Diagrama Entidade-Relacionamento (DER) projetado para a solução, detalhando as chaves primárias, estrangeiras e os relacionamentos entre as entidades.

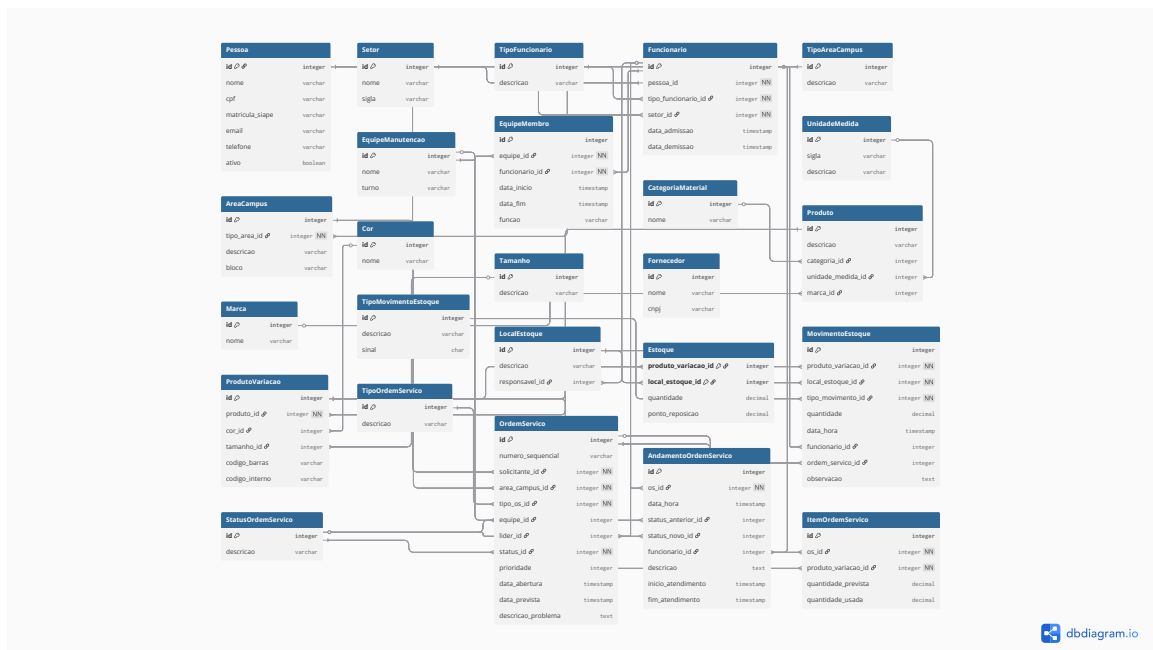


Figura 1: Diagrama Entidade-Relacionamento (DER) do SIGEJ - Parte 1.

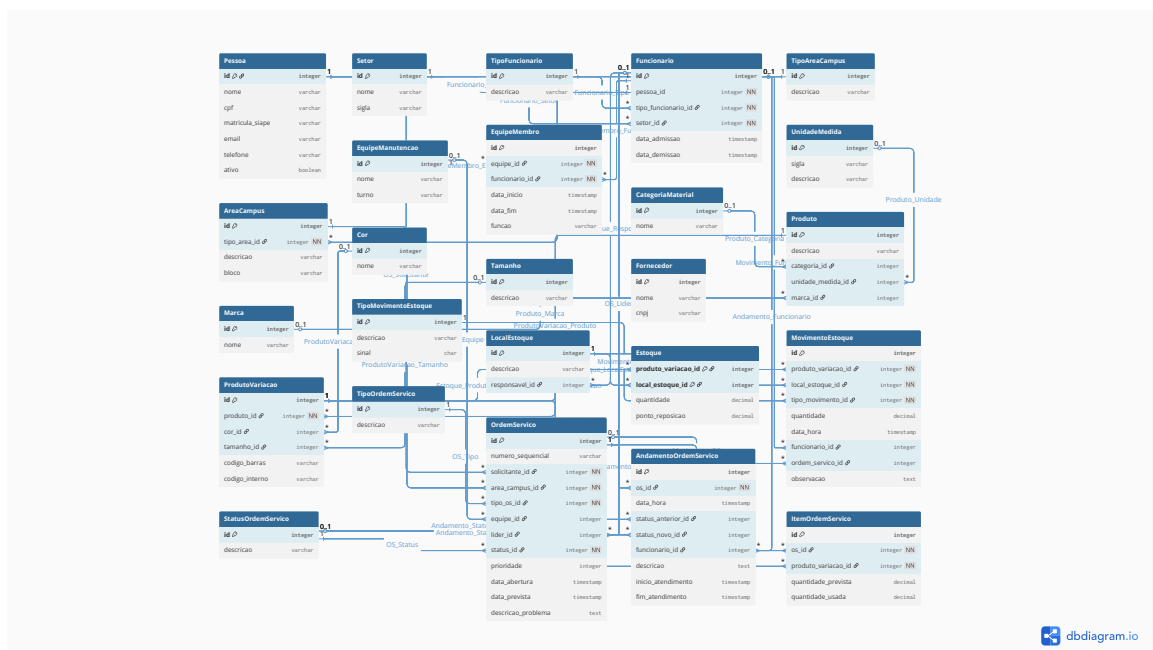


Figura 2: Diagrama Entidade-Relacionamento (DER) do SIGEJ - Parte 2.

4 Implementação do Backend e API

O sistema foi desenvolvido em C# utilizando a biblioteca nativa Npgsql para conexão com o PostgreSQL. Todas as operações de INSERT, SELECT, UPDATE e DELETE foram escritas manualmente. Abaixo, apresentamos a estrutura da API documentada via Swagger, demonstrando a organização dos endpoints desenvolvidos.

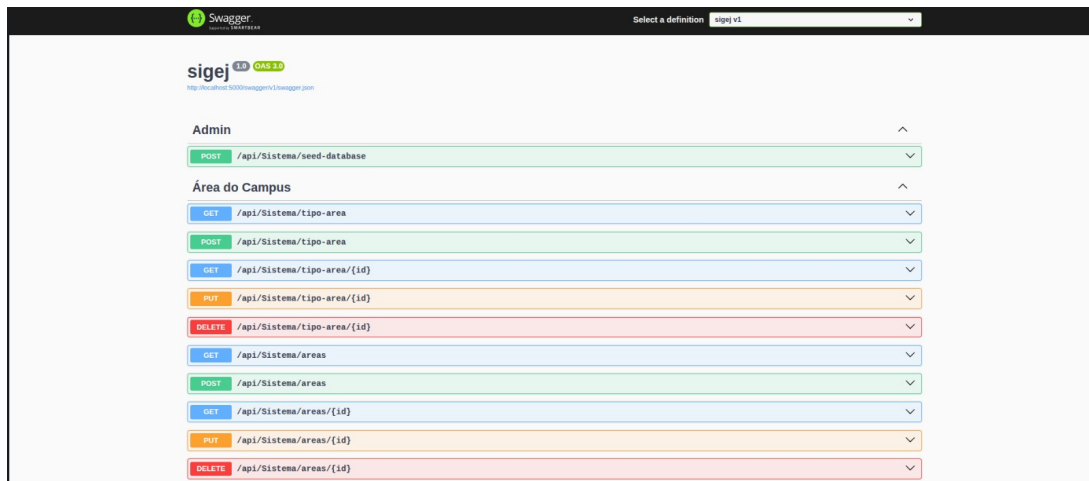


Figura 3: Endpoints de Administração e Área do Campus.

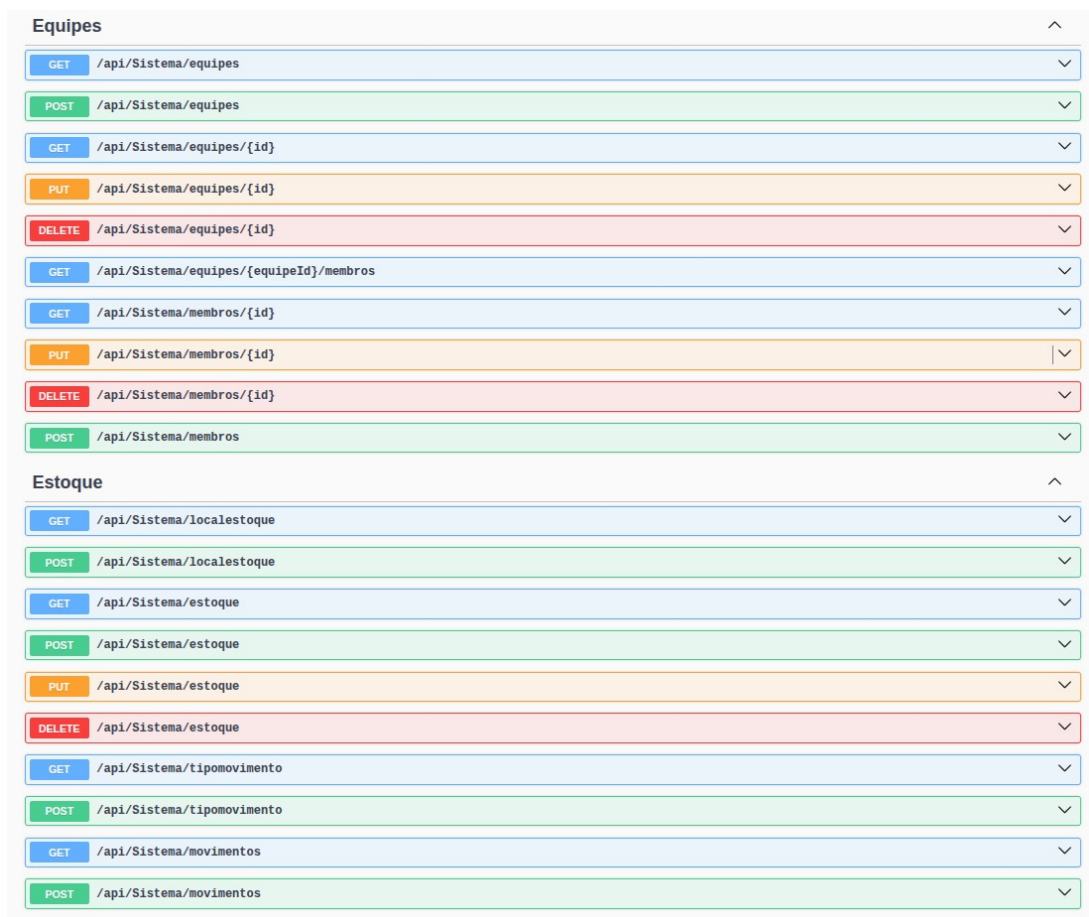


Figura 4: Endpoints de Gerenciamento de Equipes e Estoque.

Estrutura		^
GET	/api/Sistema/setores	▼
POST	/api/Sistema/setores	▼
GET	/api/Sistema/setores/{id}	▼
PUT	/api/Sistema/setores/{id}	▼
DELETE	/api/Sistema/setores/{id}	▼
Funcionários		^
GET	/api/Sistema/funcionarios	▼
POST	/api/Sistema/funcionarios	▼
GET	/api/Sistema/funcionarios/{id}	▼
PUT	/api/Sistema/funcionarios/{id}	▼
DELETE	/api/Sistema/funcionarios/{id}	▼
GET	/api/Sistema/tipo-funcionario	▼
POST	/api/Sistema/tipo-funcionario	▼
GET	/api/Sistema/tipo-funcionario/{id}	▼
PUT	/api/Sistema/tipo-funcionario/{id}	▼
DELETE	/api/Sistema/tipo-funcionario/{id}	▼

Figura 5: Endpoints de Estrutura (Setores) e Funcionários.

Ordem de Serviço		^
GET	/api/Sistema/tipos-os	▼
POST	/api/Sistema/tipos-os	▼
GET	/api/Sistema/status-os	▼
POST	/api/Sistema/status-os	▼
GET	/api/Sistema/os	▼
POST	/api/Sistema/os	▼
GET	/api/Sistema/os/{id}	▼
PUT	/api/Sistema/os/{id}	▼
DELETE	/api/Sistema/os/{id}	▼
GET	/api/Sistema/item-os	▼
POST	/api/Sistema/item-os	▼
GET	/api/Sistema/andamentos	▼
POST	/api/Sistema/andamentos	▼
Pessoas		^
GET	/api/Sistema/pessoas	▼
POST	/api/Sistema/pessoas	▼
GET	/api/Sistema/pessoas/{id}	▼
PUT	/api/Sistema/pessoas/{id}	▼
DELETE	/api/Sistema/pessoas/{id}	▼

Figura 6: Endpoints de Ordem de Serviço e Cadastro de Pessoas.

Produtos		^
GET	/api/Sistema/categorias	▼
POST	/api/Sistema/categorias	▼
GET	/api/Sistema/unidades	▼
POST	/api/Sistema/unidades	▼
GET	/api/Sistema/marcas	▼
POST	/api/Sistema/marcas	▼
GET	/api/Sistema/cores	▼
POST	/api/Sistema/cores	▼
GET	/api/Sistema/tamanhos	▼
POST	/api/Sistema/tamanhos	▼
GET	/api/Sistema/produtos	▼
POST	/api/Sistema/produtos	▼
GET	/api/Sistema/produtos/{id}	▼
PUT	/api/Sistema/produtos/{id}	▼
DELETE	/api/Sistema/produtos/{id}	▼
GET	/api/Sistema/variacoes	▼
POST	/api/Sistema/variacoes	▼

Figura 7: Endpoints de Gerenciamento de Produtos e Variações.

Relatórios		^
GET	/api/Sistema/relatorio/estoque	▼
GET	/api/Sistema/relatorio/movimentos	▼
GET	/api/Sistema/relatorio/os-abertas	▼
GET	/api/Sistema/relatorio/consumo-equipas	▼
GET	/api/Sistema/relatorio/andamentos/{ordemId}	▼

Figura 8: Lista de Endpoints de Relatórios Gerenciais.

5 Resultados e Relatórios Gerenciais

O sistema é capaz de gerar relatórios estratégicos através de consultas SQL complexas executadas diretamente pelo backend. Abaixo, demonstramos a execução das consultas obrigatórias solicitadas na especificação do projeto, evidenciadas pelas respostas JSON da API.

5.1 Execução das Consultas

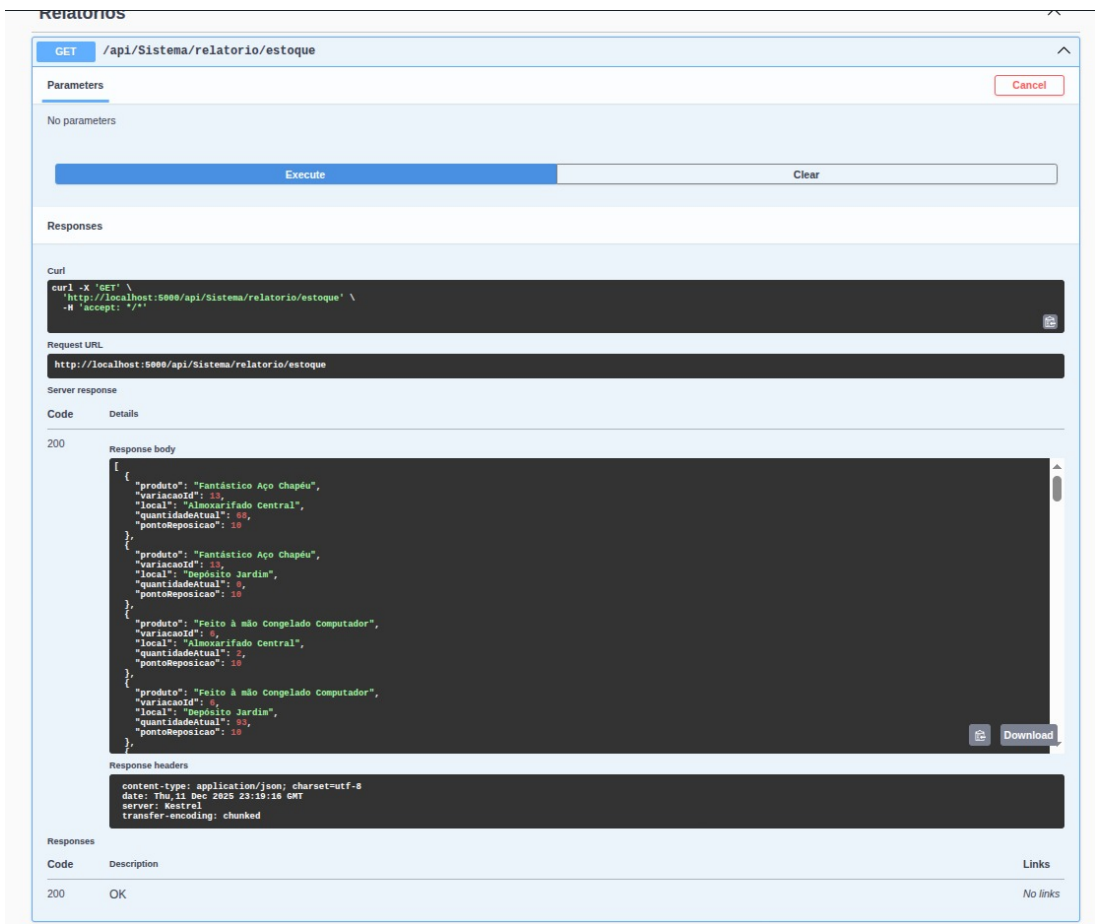


Figura 9: Resposta da API: Relatório de Estoque (JSON).

GET /api/Sistema/relatorio/movimentos

Parameters

Name	Description
inicio	
string(datetime)	2025-12-01T00:00:00Z
(query)	
fim	
string(datetime)	2025-12-11T23:59:59Z
(query)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5000/api/Sistema/relatorio/movimentos?inicio=2025-12-01T00:00:00Z&fim=2025-12-11T23:59:59Z' \
  -H 'accept: */*'
```

Request URL

http://localhost:5000/api/Sistema/relatorio/movimentos?inicio=2025-12-01T00:00:00Z&fim=2025-12-11T23:59:59Z

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "produto": "Produto q0XFD81",
  "variacaoId": 11,
  "local": "Almoxarifado Central",
  "tipoMovimento": "Saída",
  "sinal": "-1",
  "quantidade": 5,
  "datahora": "2025-12-07T19:06:00.594429",
  "funcionarioId": 11,
  "numeroOrdemService": "a3QKth"
},
{
  "id": 2,
  "produto": "Produto DQyIaght",
  "variacaoId": 22,
  "local": "Almoxarifado Central",
  "tipoMovimento": "Saída",
  "sinal": "-1",
  "quantidade": 5,
  "datahora": "2025-12-07T19:04:01.465077",
  "funcionarioId": 11,
  "numeroOrdemService": "wXOEve"
},
{
  "id": 3,
  "produto": "Produto W0raCrPq",
  "variacaoId": 11,
  "local": "Almoxarifado Central",
  "tipoMovimento": "Saída",
  "sinal": "-1",
  "quantidade": 5,
  "datahora": "2025-12-07T19:04:01.465077",
  "funcionarioId": 11,
  "numeroOrdemService": "wXOEve"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Thu, 11 Dec 2025 23:28:37 GMT
server: Nestrel
transfer-encoding: chunked
```

Responses

Code	Description
------	-------------

Links

Figura 10: Resposta da API: Relatório de Movimentos de Estoque.

GET

/api/Sistema/relatorio/os-abertas

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5000/api/Sistema/relatorio/os-abertas' \
  -H 'accept: */*'

```

Request URL

http://localhost:5000/api/Sistema/relatorio/os-abertas

Server response

Code

Details

200

Response body

```
[
  {
    "id": 8,
    "numeroSequencial": "OS-2025-108",
    "descricao": "Fecere uti omnis reprehenderit.",
    "dataAbertura": "2023-12-08T01:49:58.462894",
    "area": "Banco 72873 - Musica",
    "equipe": "Equipe transmitter"
  },
  {
    "id": 10,
    "numeroSequencial": "OS-2025-109",
    "descricao": "Mollitia eius et facilis aliquid accusamus veniam eos tempora.",
    "dataAbertura": "2024-05-11T06:16:32.545782",
    "area": "Banco 803 - Seguros & Livros",
    "equipe": "Equipe application"
  },
  {
    "id": 28,
    "numeroSequencial": "OS-2025-127",
    "descricao": "Et perspiciatis et cupiditate tenetur tenetur et sunt non.",
    "dataAbertura": "2024-06-10T08:19:35.275982",
    "area": "Banco 7248 - Eletronicos",
    "equipe": "Equipe application"
  },
  {
    "id": 13,
    "numeroSequencial": "OS-2025-112",
    "descricao": "Aut reprehenderit harum reprehenderit fugiat molestiae suscipit."
  }
]
```

Response headers

```
content-type: application/json; charset=utf-8
date: Thu, 11 Dec 2025 23:20:55 GMT
server: Kestrel
transfer-encoding: chunked

```

Responses

Code	Description	Links
200	OK	No links

Figura 11: Resposta da API: Relatório de Ordens de Serviço Abertas.

11

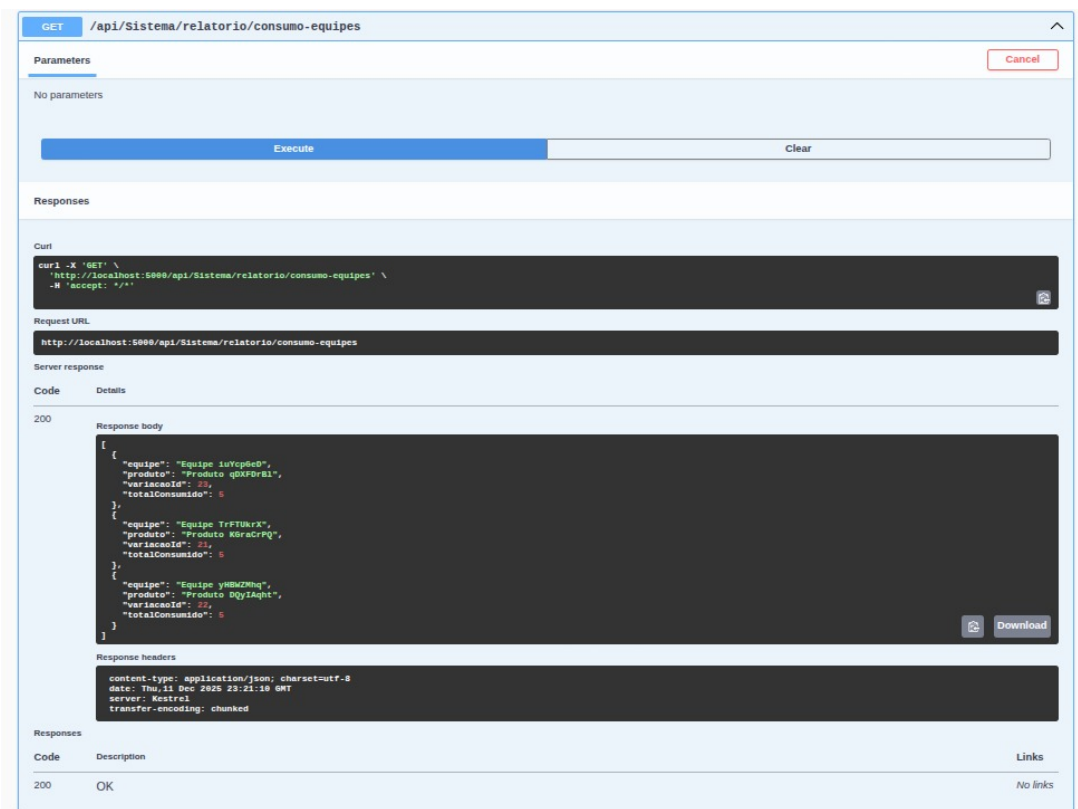


Figura 12: Resposta da API: Relatório de Consumo por Equipas.

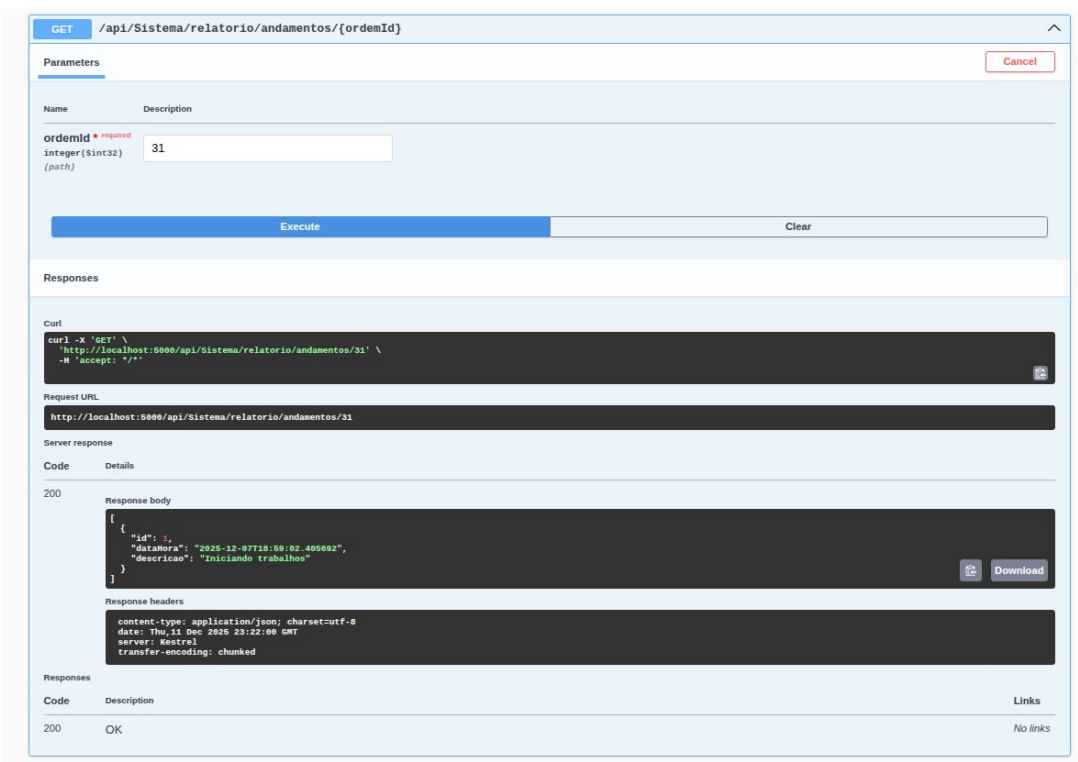


Figura 13: Resposta da API: Detalhes de Andamento da Ordem de Serviço.

6 Conclusão

O desenvolvimento do SIGEJ permitiu a aplicação prática de conceitos fundamentais de Banco de Dados e Engenharia de Software. A restrição ao uso de ORMs exigiu um conhecimento aprofundado da linguagem SQL e do funcionamento interno das conexões de banco de dados via C#.

A documentação via Swagger (apresentada nas figuras 1 a 6), o diagrama da estrutura de dados (Figura ??) e a validação das respostas JSON (figuras 7 a 11) comprovam que o sistema atende aos requisitos funcionais da Coordenação de Infraestrutura, fornecendo controle efetivo sobre manutenção e estoques do campus.

Referências

IFCE CAMPUS MARACANAÚ. **Especificação do Sistema de Gestão de Jardinagem e Manutenção - SIGEJ**. Maracanaú, 2025.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 15 Documentation**. Disponível em: <https://www.postgresql.org/>. Acesso em: dez. 2025.

MICROSOFT. **.NET Documentation (C#)**. Disponível em: <https://learn.microsoft.com/en-us/dotnet/csharp/>. Acesso em: dez. 2025.